

XRPM App Signing Implementation Guide (Direct Verification v1 - Locked)

1. Deep Link Contract

```
xrpm://signin?req=<base64url(challenge_json)>

Challenge JSON:
{
  v: 1,
  aud: https://partner.com,
  nonce: base64url_32bytes,
  iat: unix_timestamp,
  exp: unix_timestamp,
  client_id: partner_id,
  redirect_uri: https://partner.com/auth/callback,
  state: string
}
```

2. Canonical Message Format

```
XRPM_LOGIN_V1
aud=<aud>
nonce=<nonce>
iat=<iat>
exp=<exp>
client_id=<client_id>
```

3. Signing Rules

- UTF-8 encode canonical message
- Compute SHA256 digest
- Sign digest using ed25519 or secp256k1
- Signature must be returned as raw bytes base64url encoded

4. Proof Format

```
{
  v: 1,
  aud: string,
  nonce: string,
  iat: number,
  exp: number,
  client_id: string,
  account: rAddress,
  pubkey: hex,
  alg: ed25519|secp256k1,
  sig: base64url
}
```

5. Redirect Response

`redirect_uri?proof=<base64url(proof_json)>&state=<state>`

6. Eligibility Requirements

- Account must be activated (`account_info`)
- XRP balance ≥ 10 via `account_lines` trustline

7. Error Codes

`USER_REJECTED`
`EXPIRED_CHALLENGE`
`INVALID_REDIRECT`
`NOT_ELIGIBLE`

8. Security Requirements

- Private keys never leave device
- Use secure storage (Keychain/Keystore)
- Display domain clearly
- Block invalid `redirect_uri`
- Enforce HTTPS

9. Implementation Checklist

- ☐ Deep link handler
- ☐ Challenge parser
- ☐ Consent UI
- ☐ Canonical builder
- ☐ SHA256 hashing
- ☐ Signing (ed25519 + secp256k1)
- ☐ Proof generation
- ☐ Callback redirect
- ☐ Eligibility pre-check